LeetCode Problem #3

The **Longest Substring Without Repeating Characters** is a common problem on LeetCode (**Problem #3**). It requires finding the length of the longest substring in a given string where no character is repeated.

Problem Statement

Given a string `s`, find the length of the **longest substring** without repeating characters.

Example 1

```
Input: s = "abcabcbb"
Output: 3
Explanation: The longest substring is "abc" with length 3.
```

Example 2

```
Input: s = "bbbbb"
Output: 1
Explanation: The longest substring is "b" with length 1.
```

Example 3

```
Input: s = "pwwkew"
Output: 3
Explanation: The longest substring is "wke" with length 3.
```

Optimal Solution: Sliding Window Approach

We use a **Sliding Window with HashSet** (or HashMap) to keep track of characters and their positions.

Steps

1. Use two pointers: `left` and `right` to define a window.
2. Move `right` to expand the window while adding characters to a **HashSet**.
3. If a character repeats, move `left` forward to remove it from the window.
4. Keep track of the maximum window size.

Java Code: Sliding Window

```
import java.util.*;

class LongestSubstring {
```

```
    public int lengthOfLongestSubstring(String s) {
        Set<Character> set = new HashSet<>();
        int left = 0, maxLength = 0;

        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }

    public static void main(String[] args) {
        LongestSubstring obj = new LongestSubstring();
        System.out.println(obj.lengthOfLongestSubstring("abcabcbb")); //
Output: 3
        System.out.println(obj.lengthOfLongestSubstring("bbbbb"));    //
Output: 1
        System.out.println(obj.lengthOfLongestSubstring("pwwkew"));   //
Output: 3
    }
}
```

Time Complexity Analysis

- **O(n)**: Each character is processed once (added and removed at most once).
- **O(min(n, m))** space: `m` is the number of unique characters.

Alternative Approach: HashMap

We can optimize the sliding window further using a **HashMap** to store character indices.

Code Using HashMap

```
import java.util.*;

class LongestSubstringMap {
```

```
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> map = new HashMap<>();
        int left = 0, maxLength = 0;

        for (int right = 0; right < s.length(); right++) {
            if (map.containsKey(s.charAt(right))) {
                left = Math.max(map.get(s.charAt(right)) + 1, left);
            }
            map.put(s.charAt(right), right);
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }

    public static void main(String[] args) {
        LongestSubstringMap obj = new LongestSubstringMap();
        System.out.println(obj.lengthOfLongestSubstring("abcabcbb")); //
Output: 3
        System.out.println(obj.lengthOfLongestSubstring("bbbbb"));    //
Output: 1
        System.out.println(obj.lengthOfLongestSubstring("pwwkew"));   //
Output: 3
    }
}
```

Key Takeaways

1. **Sliding Window + HashSet**: Works well but needs `O(n)` space.
2. **Sliding Window + HashMap**: Faster because it skips duplicate characters instead of moving `left` one step at a time.
3. **Time Complexity**: **O(n)**
4. **Space Complexity**: **O(min(n, m))**, where `m` is the number of unique characters.

This is an essential problem for **string manipulation and sliding window technique**, commonly asked in coding interviews. 🚀